Algorithm Engineering
Woche 2, Aufgabe 4

Jan Vogt

π mit #pragma omp for

In `pi_omp_for.cpp` ist das Programm `pi_numerical_integration.cpp` aus der letzten Vorlesung mit `#pragma omp for` parallelisiert. Die Schleife wird von OpenMP auf die Threads aufgeteilt, jeder Thread summiert in eine lokale Variable und am Ende wird diese mit `#pragma omp atomic` zur gemeinsamen Summe addiert:

```
#pragma omp parallel num_threads(num_threads)
{
  double sum_local = 0.0;
#pragma omp for schedule(runtime)
  for (int i = 0; i < num_steps; i++) {
    double x = (i + 0.5) * width;
    sum_local = sum_local + (1.0 / (1.0 + x * x));
  }
#pragma omp atomic
  sum += sum_local;
}
```

Mit `schedule(runtime)` kann man den Schedule beim Starten über die Umgebungsvariable festlegen, zum Beispiel `OMP_SCHEDULE=static ./pi_omp_for 8`. Die Threadanzahl wird als Argument übergeben.

Messung

Gemessen habe ich auf meinem Laptop (Apple M3) mit 100.000.000 Schritten, ohne Optimierung und jeweils 5 Läufen. In der Tabelle steht der Median in Sekunden.

Schedule	1 Thread	2 Threads	4 Threads	8 Threads
`static`	0.1401	0.0709	0.0372	0.0314
`dynamic, 1`	0.5664	2.9519	2.3110	4.1473
`dynamic, 1000`	0.1404	0.0720	0.0371	0.0286
`dynamic, 100000`	0.1400	0.0710	0.0365	0.0273
`guided`	0.1400	0.0710	0.0368	0.0273

Mit `dynamic, 1` ist das Programm mit mehreren Threads viel langsamer als mit einem, weil sich die Threads für jede einzelne der 100 Millionen Iterationen neue Arbeit holen müssen. Ohne `OMP_SCHEDULE` waren die Zeiten bei mir genauso schlecht wie mit `dynamic, 1`, man sollte die Variable bei `schedule(runtime)` also immer setzen.

Bis 4 Threads sind die anderen Schedules ungefähr gleich schnell, weil jede Iteration gleich viel Arbeit ist. Mit 8 Threads sind `dynamic, 100000` und `guided` aber etwa 13 Prozent schneller als `static`. Das ist mir auch im Zusammenhang mit Woche 1 aufgefallen: Dort mussten bei der statischen Aufteilung vermutlich die Threads auf den Performance-Kernen auf die langsameren Effizienz-Kerne warten. Mit `dynamic` können sich die schnellen Kerne einfach mehr Blöcke holen. Das passt zu Folie 19, wo steht, dass `static` bei Effizienz-Kernen nicht unbedingt die beste Wahl ist.

Außerdem war π bei `static` und gleicher Threadanzahl fast immer exakt gleich, bei `dynamic` dagegen in den letzten Stellen bei jedem Lauf etwas anders, weil die Blöcke jedes Mal anders auf die Threads verteilt werden.